

Lab 20 - expected output

Reference transcript for Lab 20 | Control Plane Anatomy and etcd Backup/Restore, namespace kcna-lab20.

How to read this file. Node names, container IDs, config hashes, lease holder identities, IP addresses, etcd revisions, key counts, snapshot sizes and timestamps are environment-specific and will differ. Object names, annotation keys, flag names, counts derived from data/, and the safety-relevant strings must match.

>

Everything shown here is read-only against the control plane. No output in this file was produced by a command that modifies cluster state, except the deliberate mirror-Pod delete in Failure Injection A, whose whole point is that it changes nothing.

4.1 - datasets

```
$ echo "rows=$(( $(wc -l < data/control-plane-inventory.csv) - 1 ))"
rows=9
$ echo "static_pods=$(grep -c ',StaticPod,' data/control-plane-inventory.csv)"
static_pods=4
```

These two numbers are load-bearing - the Job in Step 4.7 and the graded checks both compare against them.

4.2 - the control plane

```
$ kubectl -n kube-system get pods -o wide
NAME                                READY   STATUS    RESTARTS   AGE   IP
NODE
coredns-7db6d8ff4d-lc4nv            1/1     Running   0           96m   10.244.0.3
kcna-control-plane
coredns-7db6d8ff4d-w8m2j            1/1     Running   0           96m   10.244.0.2
kcna-control-plane
etcd-kcna-control-plane             1/1     Running   0           96m   172.18.0.2
kcna-control-plane
kindnet-8xq6r                      1/1     Running   0           96m   172.18.0.2
kcna-control-plane
kube-apiserver-kcna-control-plane    1/1     Running   0           96m   172.18.0.2
kcna-control-plane
kube-controller-manager-kcna-control-plane 1/1     Running   0           96m   172.18.0.2
kcna-control-plane
kube-proxy-2wr7n                   1/1     Running   0           96m   172.18.0.2
kcna-control-plane
kube-scheduler-kcna-control-plane    1/1     Running   0           96m   172.18.0.2
kcna-control-plane
```

Required signal: the four control-plane Pods and kube-proxy/kindnet share the node IP (hostNetwork: true); CoreDNS has a Pod-network IP. The specific addresses vary.

```
$ kubectl get --raw='/readyz?verbose'
[+]ping ok
[+]log ok
[+]etcd ok
[+]etcd-readiness ok
[+]informer-sync ok
...
readyz check passed
```

Required signal: [+]etcd ok and a final readyz check passed. The number and names of the poststarthook/ checks vary by minor version - do not treat their absence as a fault.

4.3 - static Pods and mirror Pods

```
$ kubectl -n kube-system get pod "etcd-$NODE" -o jsonpath='{.metadata.ownerReferences[0].kind}/{.metadata.ownerReferences[0].name}{"\n"}'
Node/kcna-control-plane
```

Required signal: owner kind is Node. Any other owner kind means it is not a static Pod.

```
$ kubectl -n kube-system get pod "etcd-$NODE" -o jsonpath='{.metadata.annotations}{"\n"}' | tr
',' '\n' | grep config
"kubernetes.io/config.hash":"9a4c1f7e0b3d2856ac9f4b17e05d3c82"
"kubernetes.io/config.mirror":"9a4c1f7e0b3d2856ac9f4b17e05d3c82"
"kubernetes.io/config.seen":"2026-09-05T03:11:02.417392514Z"
"kubernetes.io/config.source":"file"
```

Required signal: "kubernetes.io/config.source":"file". The hashes and timestamp vary.

```
$ kubectl -n kube-system get pods -o jsonpath='{range .items[?(@.metadata.annotations.
kubernetes.io/config.source=="file")]}{.metadata.name}{"\n"}{end}'
etcd-kcna-control-plane
kube-apiserver-kcna-control-plane
kube-controller-manager-kcna-control-plane
kube-scheduler-kcna-control-plane
```

Required signal: exactly four names, matching static_pods=4 from the dataset.

4.4 / 4.5 - the manifest directory and the etcd manifest

```
$ docker exec "$NODE" ls -l /etc/kubernetes/manifests
total 16
-rw----- 1 root root 2406 Sep  5 03:10 etcd.yaml
-rw----- 1 root root 3896 Sep  5 03:10 kube-apiserver.yaml
-rw----- 1 root root 3428 Sep  5 03:10 kube-controller-manager.yaml
-rw----- 1 root root 1463 Sep  5 03:10 kube-scheduler.yaml
```

File sizes vary between versions. Four files is the signal.

```
$ kubectl -n kube-system get pod "etcd-$NODE" -o yaml | grep -E 'image:|--(advertise-client-
urls|cert-file|key-file|trusted-ca-file|data-dir|listen-client-urls|listen-peer-urls|initial-
cluster)= '
  --advertise-client-urls=https://172.18.0.2:2379
  --cert-file=/etc/kubernetes/pki/etcd/server.crt
  --data-dir=/var/lib/etcd
  --initial-cluster=kcna-control-plane=https://172.18.0.2:2380
  --key-file=/etc/kubernetes/pki/etcd/server.key
  --listen-client-urls=https://127.0.0.1:2379,https://172.18.0.2:2379
  --listen-peer-urls=https://172.18.0.2:2380
  --trusted-ca-file=/etc/kubernetes/pki/etcd/ca.crt
image: registry.k8s.io/etcd:3.5.15-0
```

Required signal: ports 2379 (client) and 2380 (peer); the three PKI paths under /etc/kubernetes/pki/etcd/; --data-dir=/var/lib/etcd. The image tag will differ on your cluster and determines whether you use etcdctl or etcdctl in Step 4.9.

4.6 - leader election

```
$ kubectl -n kube-system get leases
NAME                                     HOLDER
apiserver-lz3rd7qmtqvfbhvxwl4gk6nz2a  apiserver-lz3rd7qmtqvfbhvxwl4gk6nz2a_0f1d6c9b-...
99m
kube-controller-manager                 kcna-control-plane_3b8e5d21-6c47-4a09-9e13-5f0c82ab7d64
99m
kube-scheduler                          kcna-control-plane_c7d0a493-8f52-41be-b6a7-2e9d5310cf88
99m
```

Required signal: kube-controller-manager and kube-scheduler leases exist with a holder. Holder identity strings and the apiserver identity lease name are random.

```
$ kubectl -n kcna-lab20 get events --field-selector involvedObject.name=trace-probe -o custom-
columns='TIME:.lastTimestamp,SOURCE:.source.component,REASON:.reason,MESSAGE:.message'
TIME                SOURCE              REASON    MESSAGE
2026-09-05T06:55:12Z default-scheduler   Scheduled  Successfully assigned kcna-lab20/trace-
probe to kcna-control-plane
2026-09-05T06:55:13Z kubelet            Pulled    Container image "busybox:1.36" already
present on machine
2026-09-05T06:55:13Z kubelet            Created   Created container: trace-probe
2026-09-05T06:55:13Z kubelet            Started   Started container trace-probe
```

Required signal: Scheduled from default-scheduler, then Created/Started from kubelet. Two components, no direct contact. Pulled may read Pulling image ... on a first run.

4.7 - the concrete inventory

```
$ kubectl -n kcna-lab20 logs job/cp-inventory
TrackLane control-plane inventory for node kcna-control-plane
generated from data/control-plane-inventory.csv
=====
COMPONENT          DEPLOYMENT  OBJECT OR PATH TO INSPECT
kube-apiserver      StaticPod   kube-system/pod/kube-apiserver-kcna-control-plane
etcd                 StaticPod   kube-system/pod/etcd-kcna-control-plane
kube-controller-manager
plane               StaticPod   kube-system/pod/kube-controller-manager-kcna-control-
plane
kube-scheduler      StaticPod   kube-system/pod/kube-scheduler-kcna-control-plane
kubelet             SystemdUnit /etc/systemd/system/kubelet.service.d/10-kubeadm.conf
kube-proxy          DaemonSet   api-object:kube-system/daemonset/kube-proxy
coredns             Deployment  api-object:kube-system/deployment/coredns
kindnet             DaemonSet   api-object:kube-system/daemonset/kindnet
local-path-provisioner
provisioner         Deployment  api-object:local-path-storage/deployment/local-path-
provisioner
=====
inventory_rows=9
static_pod_components=4
namespace=kcna-lab20
NOTE: this Job has namespaced RBAC only and cannot read kube-system.
```

Required signal: inventory_rows=9, static_pod_components=4, and the four StaticPod rows resolved to <component>-<your node name>.

```
$ kubectl auth can-i list pods --namespace=kube-system --as=system:serviceaccount:kcna-lab20:
cp-inventory
no
$ kubectl auth can-i list pods --namespace=kcna-lab20 --as=system:serviceaccount:kcna-lab20:cp-
inventory
yes
```

Required signal: exactly no then yes. A yes on the first would mean the Role was widened to a ClusterRole and the least-privilege lesson has been lost.

4.8 - snapshot save (NON-DESTRUCTIVE)

```
$ kubectl -n kube-system exec "$ETCD_POD" -- etcdctl version
etcdctl version: 3.5.15
API version: 3.5
```

```
$ kubectl -n kube-system exec "$ETCD_POD" -- etcdctl \
  --endpoints=https://127.0.0.1:2379 \
  --cacert=/etc/kubernetes/pki/etcd/ca.crt \
  --cert=/etc/kubernetes/pki/etcd/server.crt \
  --key=/etc/kubernetes/pki/etcd/server.key \
  snapshot save /var/lib/etcd/kcna-lab20-snapshot.db
{"level":"info",...,"msg":"created temporary db file","path":"/var/lib/etcd/kcna-lab20-
snapshot.db.part"}
{"level":"info",...,"msg":"opened snapshot stream; downloading"}
{"level":"info",...,"msg":"fetching snapshot","endpoint":"https://127.0.0.1:2379"}
{"level":"info",...,"msg":"completed snapshot read; closing"}
{"level":"info",...,"msg":"fetched snapshot","endpoint":"https://127.0.0.1:2379","size":"7.4
MB","took":"now"}
{"level":"info",...,"msg":"saved","path":"/var/lib/etcd/kcna-lab20-snapshot.db"}
Snapshot saved at /var/lib/etcd/kcna-lab20-snapshot.db
```

Required signal: the final line Snapshot saved at <path> and exit code 0. The JSON log lines are written to stderr and their field ordering varies. size depends on how much you have created in Labs 01-19.

The cluster is unaffected - confirm immediately after:

```
$ kubectl get --raw='/readyz' && echo
ok
```

4.9 - snapshot status (NON-DESTRUCTIVE)

On etcd 3.5.x and later, etcdctl is the correct binary:

```
$ kubectl -n kube-system exec "$ETCD_POD" -- etcdctl snapshot status /var/lib/etcd/kcna-lab20-
snapshot.db -w table
+-----+-----+-----+-----+
|  HASH   | REVISION | TOTAL KEYS | TOTAL SIZE |
+-----+-----+-----+-----+
| a41f7b2e |    38412 |        1187 |    7.4 MB |
+-----+-----+-----+-----+
```

Required signal: a four-column table with a non-empty HASH, a REVISION greater than 0, TOTAL KEYS in the hundreds or thousands, and a TOTAL SIZE of at least a few MB. All four values will differ on your cluster - what matters is that none of them is zero.

If etcdctl is not in the image (etcd older than 3.5):

```
OCI runtime exec failed: exec failed: unable to start container process: exec: "etcdctl":
executable file not found in $PATH: unknown
command terminated with exit code 126
```

Then the deprecated form gives the same table:

```
$ kubectl -n kube-system exec "$ETCD_POD" -- etcdctl snapshot status /var/lib/etcd/kcna-lab20-
snapshot.db -w table
Deprecated: Use `etcdctl snapshot status` instead.
+-----+-----+-----+-----+
|  HASH   | REVISION | TOTAL KEYS | TOTAL SIZE |
+-----+-----+-----+-----+
| a41f7b2e |    38412 |        1187 |    7.4 MB |
+-----+-----+-----+-----+
```

Copying the file off the node:

```
$ docker exec "$NODE" ls -lh /var/lib/etcd/kcna-lab20-snapshot.db
-rw----- 1 root root 7.1M Sep  5 07:02 /var/lib/etcd/kcna-lab20-snapshot.db

$ docker cp "$NODE:/var/lib/etcd/kcna-lab20-snapshot.db" /tmp/kcna-lab20-snapshot.db
Successfully copied 7.42MB to /tmp/kcna-lab20-snapshot.db
```

4.11 - reading the runbook from inside the cluster

```
$ kubectl -n kcna-lab20 logs runbook-reader
=== SAFETY BANNER FROM THE RUNBOOK ===
10:## ■ DO NOT RUN ANY COMMAND IN SECTION 3 ON A SHARED OR CLASSROOM CLUSTER
75:## 3. ■ RESTORE - DESTRUCTIVE. DO NOT RUN ON A SHARED CLUSTER ■

=== RECOVERY OBJECTIVES FROM THE BACKUP POLICY ===
rpo_minutes: 30          # max acceptable data loss
rto_minutes: 60          # decision -> /readyz ok
interval: "*/30 * * * *"
offsite_required: true    # object storage, different failure domain

=== WHAT AN etcd RESTORE DOES *NOT* RECOVER ===
### What restore does NOT recover
...
=== SECTION COUNT ===
runbook_headings=6
do_not_run_banners=2
```

Required signal: runbook_headings=6 and do_not_run_banners=2, matching data/etcd-restore-runbook.md. Line numbers 10: and 75: shift if the runbook is edited; the two banner texts must both be present.

Housekeeping leaves the data directory as it was:

```
$ docker exec "$NODE" rm -f /var/lib/etcd/kcna-lab20-snapshot.db
$ docker exec "$NODE" ls /var/lib/etcd
member
```

Required end state: only member/ - etcd's real data directory, never touched at any point in this lab.

Graded checks

```
$ bash verification/checks.sh
== Lab 20 verification - namespace kcna-lab20 ==
[PASS] namespace kcna-lab20 exists
[PASS] control plane is ready (/readyz reports ok)
[PASS] apiserver reports [+]etcd ok in /readyz?verbose
[PASS] 4 static-pod components found with config.source=file
[PASS] mirror pod etcd-kcna-control-plane is owned by a Node, not a controller
[PASS] etcd static pod exposes client port 2379 and peer port 2380
[PASS] configmap control-plane-inventory holds 9 data rows (matches data/)
[PASS] configmap etcd-restore-runbook carries 2 DO-NOT-RUN banners
[PASS] job/cp-inventory completed and reported static_pod_components=4
[PASS] serviceaccount cp-inventory CANNOT list pods in kube-system
[PASS] serviceaccount cp-inventory CAN list pods in kcna-lab20
[PASS] configmap etcd-backup-policy declares rpo_minutes and rto_minutes
[PASS] pod/runbook-reader runs with automountServiceAccountToken=false
[PASS] pod/runbook-reader has no hostPath volumes
[PASS] etcdctl is available inside the etcd static pod (etcd-kcna-control-plane)
[PASS] etcd endpoint status reports revision 38412 over TLS (read-only)
[PASS] no container in kcna-lab20 requests privileged: true
[PASS] no cluster-scoped object was created by this lab
-----
18 passed, 0 failed
```

Exit code 0. The reported revision will differ.

Safety properties of this script, which you should confirm by reading it: its only etcd calls are etcdctl version and etcdctl endpoint status. It does not call endpoint health (which commits a no-op Raft proposal), snapshot restore, defrag, compact or move-leader. It writes no file anywhere.

Failure injection A - deleting a mirror Pod does nothing

```
$ kubectl -n kube-system delete pod "kube-scheduler-$NODE"
pod "kube-scheduler-kcna-control-plane" deleted

$ sleep 8; kubectl -n kube-system get pod "kube-scheduler-$NODE"
NAME                                READY   STATUS    RESTARTS   AGE
kube-scheduler-kcna-control-plane  1/1     Running   0           7s
```

Required signal: the Pod is back, 1/1 Running, with a fresh AGE and RESTARTS 0. The API object is new; the container never stopped. This is safe and is the point - you cannot stop a static Pod through the API.

Failure injection B - reading a control-plane outage

Healthy (what you see on this cluster):

```
$ kubectl get --raw='/readyz?verbose'
...
[+]etcd ok
...
readyz check passed
```

Unhealthy (reference only - not reproduced on the class cluster):

```
[+]ping ok
[+]log ok
[-]etcd failed: reason withheld
[-]etcd-readiness failed: reason withheld
[+]informer-sync ok
...
readyz check failed
```

Required signal to recognise: a [-] prefix names the failing subsystem. [-]etcd failed points at storage; the diagnostic ladder in Section 6 of the README continues from there. reason withheld is deliberate - health endpoints are pre-auth and must not leak internal detail.

Cleanup

```
$ kubectl delete namespace kcna-lab20 --wait=true
namespace "kcna-lab20" deleted

$ docker exec "$NODE" ls /var/lib/etcd
member

$ kubectl get --raw='/readyz' && echo
ok

$ kubectl -n kube-system get pods --no-headers | wc -l
8

$ kubectl get namespace kcna-lab20
Error from server (NotFound): namespaces "kcna-lab20" not found
```

Required end state: /readyz still ok, the same number of kube-system Pods as before the lab, /var/lib/etcd containing only member/, and no kcna-lab20 namespace. This lab creates no cluster-scoped objects and modifies no control-plane object.